

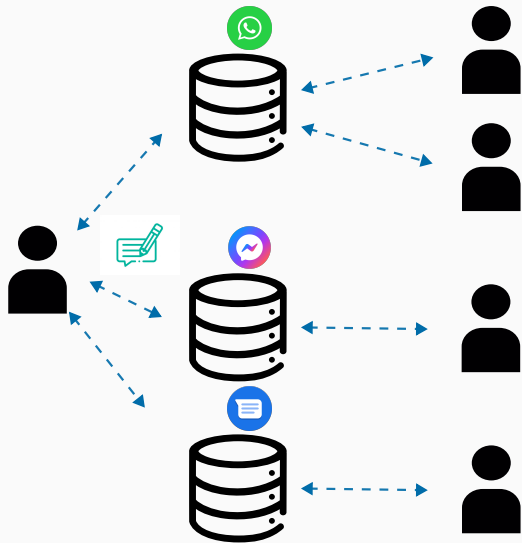
SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments

Ruben Eschauzier, Ruben Taelman

September 30, 2026

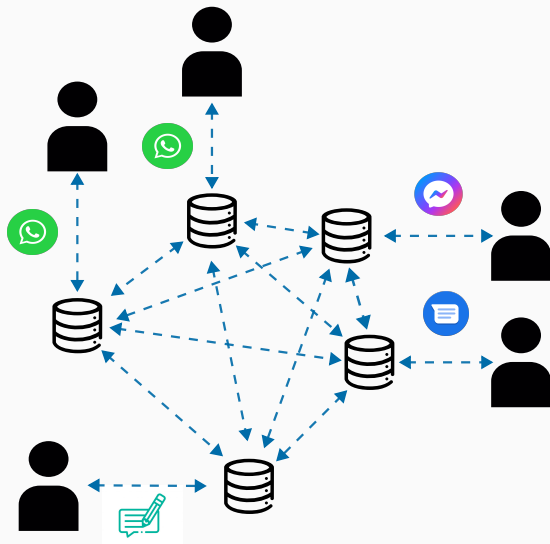
Department of Electronics and Information Systems, Ghent University Imec

The Motivation for Decentralized Personal Data Storage



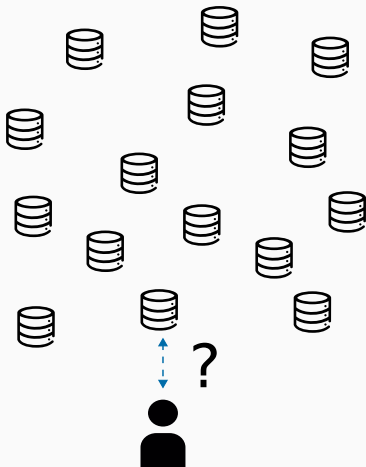
- Each application has its own data
- Stifles innovation
- Causes vendor lock-in

The Motivation for Decentralized Personal Data Storage



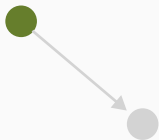
- Each application accesses a common decentralized data environment
- Easy to switch vendors
- Promotes innovation

Querying Structured Decentralized Environments



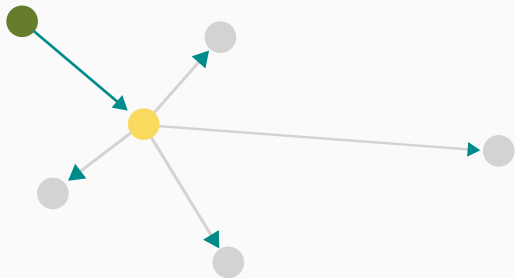
- How do we discover these sources?
- How do we ensure fine-grained access control?

Link Traversal: Seed Document



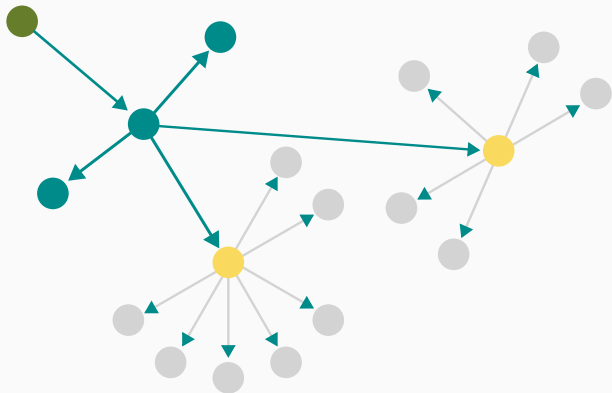
- Link Traversal starts from seed documents (URIs)
- These are provided by the user or specified in the query

Link Traversal: Traversal



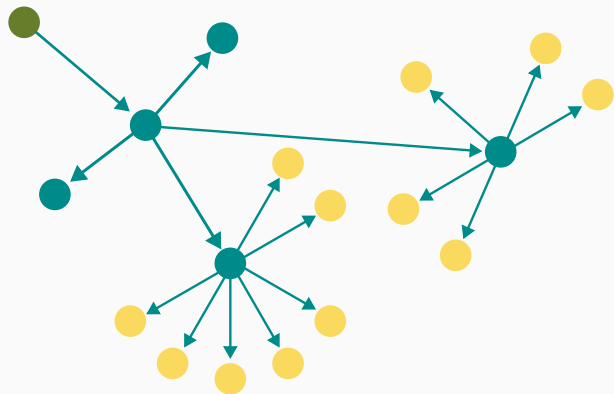
- New URIs are extracted from the seed document
- URIs are extracted in accordance with reachability criteria
- Uses the follow-your-nose principle

Link Traversal: Traversal



- New URIs are dereferenced and the process is repeated

Link Traversal: Termination

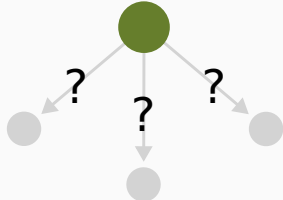


- This continues until all links are dereferenced
- Continuously produces results
- Can enforce fine-grained (document-level) access control

Query Optimization for Link Traversal

```
SELECT * WHERE {  
  <seedUri> <ex:p1> ?o1.  
  <seedUri> <ex:p2> ?o2.  
  ...  
}
```

Extract
Seed document

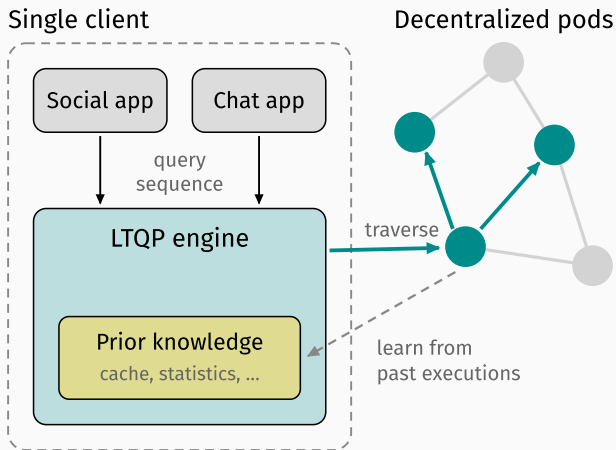


The query (partly) determines:

- The queried data
- The topology of the queried data
- The query-relevant documents

Result: Limited prior knowledge for query optimization

Query Optimization for Link Traversal: Client-side Engines



- Engine instances serve a single client
- Can we exploit client-specific usage patterns and ‘personalize’ optimization
- Gain prior knowledge by looking at past executions

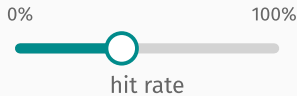
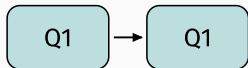
Problem Statement - Evaluating Client-Specific Usage Patterns

- How do we evaluate these approaches?
- Real-world query sequences issued to decentralized applications
- In their absence, what are the alternatives?

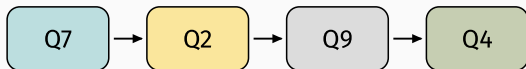
Evaluating the performance of client-specific optimization techniques in realistic, user-specific scenarios

Existing Evaluation Approaches

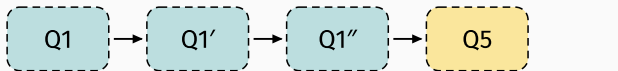
Micro benchmark



Random grouping



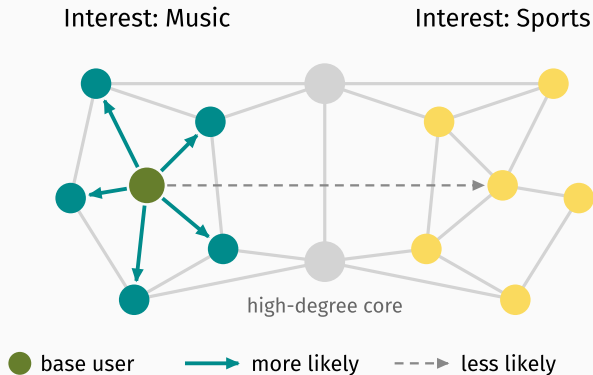
Real-world query log



- Micro benchmarks isolate single variables (Hartig 2011; Nicholson et al. 2024)
- End-to-end benchmarks group unrelated queries (Bizer and Schultz 2011; Martin et al. 2010)
- Real-world query logs are the gold standard (Saleem et al. 2015), but none exist for LTQP

- Next best option: a unified simulated benchmark
- Common basis for comparison and improved reproducibility
- Grounded in usage patterns observed in the literature:
 - User interest-based query behavior
 - Logical query sessions
 - Query refinement

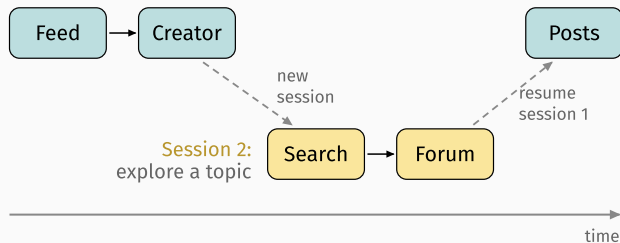
Usage Patterns: User Interest-Based Query Behavior



- Users form communities based on shared interests (Mislove et al. 2007)
 - Shared interests drive more interactions between users (Mitzlaff et al. 2013)
- ⇒ Simulate user-relatedness when choosing what to query

Usage Patterns: Logical Query Sessions

Session 1: follow a creator



- Browsing is grouped into goal-oriented logical sessions (Meiss et al. 2009)
 - New sessions begin in approximately 15% of clicks
 - Next query builds on previous results
- ⇒ Simulate dependent queries and session switching

SPARQL query logs (Bonifati et al. 2020)

- Related queries are issued in streaks
- Iteratively refined towards a shared goal
- Mostly adding or removing **FILTER**, **UNION**, and **OPTIONAL**

Exploratory querying (Arenas et al. 2025)

- Result refinement
- Result generalization
- Result extension
- Bug fixing
- Query refinement
- Shifting query intent

⇒ Queries evolve step-by-step instead of being independent templates

Usage Patterns: Query Refinement Example

Base template

```
?msg :hasCreator <alice> .  
?msg :content ?text .
```

Result refinement: + FILTER

```
?msg :hasCreator <alice> .  
?msg :content ?text .  
FILTER(CONTAINS(?text, "jazz"))
```

Shifting query intent: new seed

```
?msg :hasCreator <bob> .  
?msg :content ?text .  
FILTER(CONTAINS(?text, "jazz"))  
OPTIONAL { ?msg :hasTag ?tag }
```

Result extension: + OPTIONAL

```
?msg :hasCreator <alice> .  
?msg :content ?text .  
FILTER(CONTAINS(?text, "jazz"))  
OPTIONAL { ?msg :hasTag ?tag }
```

Simulated Solid Environment: SolidBench (Taelman and Verborgh 2023)

- Data is stored in vaults simulating social media application

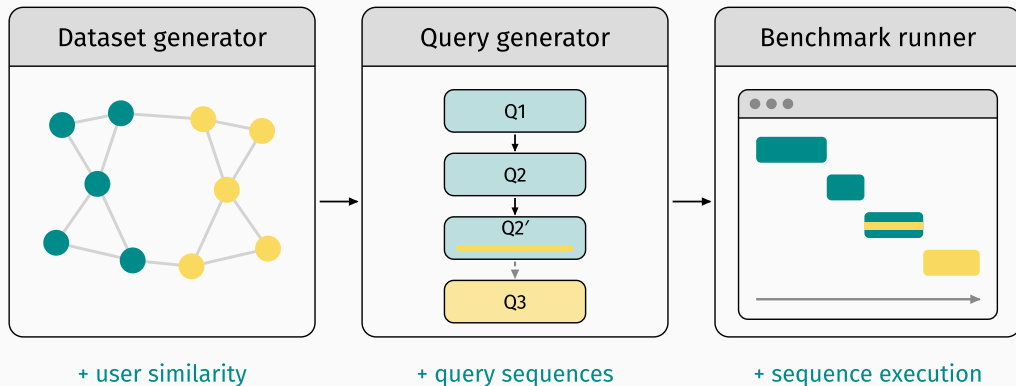
Data attributes

- Generates 158,233 RDF files
- Across 1,531 data vaults
- containing a total of 3,556,159 triples.

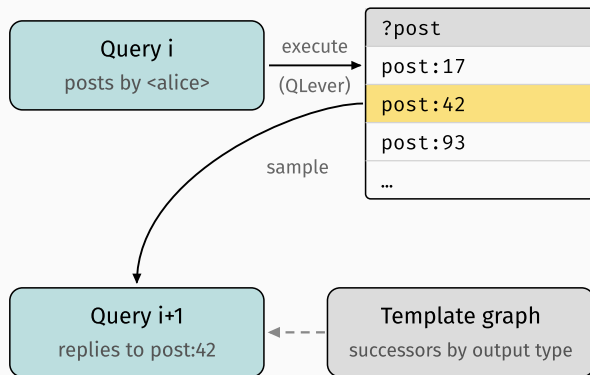
Analysed queries

- Use discover queries

Simulation Methodology: Extending SolidBench



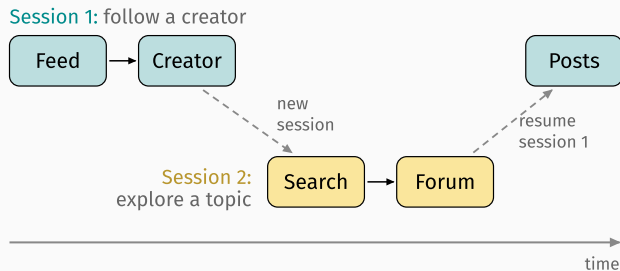
Simulation Methodology: Logical Query Sessions



No results → fall back to interest-based instantiation

- Directed template graph: output types map to valid next templates
- Instantiation values sampled from previous results
- Simulates “click-through” behavior

Simulation Methodology: Session Switching



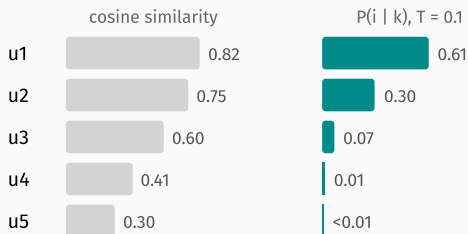
- Switch probability per sequence: log-normal, mean 15% (Meiss et al. 2009)
- Switch: new session or resume a previous one (50/50)
- No backtracking, balanced template distribution

Simulation Methodology: User Interest Modeling

Base user k



Top-M similar users



- Binary vector of interests and studies per user
- Cosine similarity to base user k, keep top-M
- Softmax selects the first query of each session:

$$P(i | k) = \frac{e^{s_{k,i}/T}}{\sum_{u \in C_{k,M}} e^{s_{k,u}/T}}$$

Behavior	Transformation	Choke points
Result refinement	$\pm$ FILTER , $\pm$ selective triple patterns	P.1, P.3
Result generalization	$+$ (non-)selective triple patterns	P.3, P.4
Result extension	$+$ triple patterns, $+$ OPTIONAL/UNION	P.2–P.4
Query refinement	$\pm$ OPTIONAL/UNION	P.2
Shifting query intent	New instantiation values (seeds)	T.1, T.2

P.1 Filters

P.2 Filter push-down through **OPTIONAL/UNION**

P.3 Selective patterns

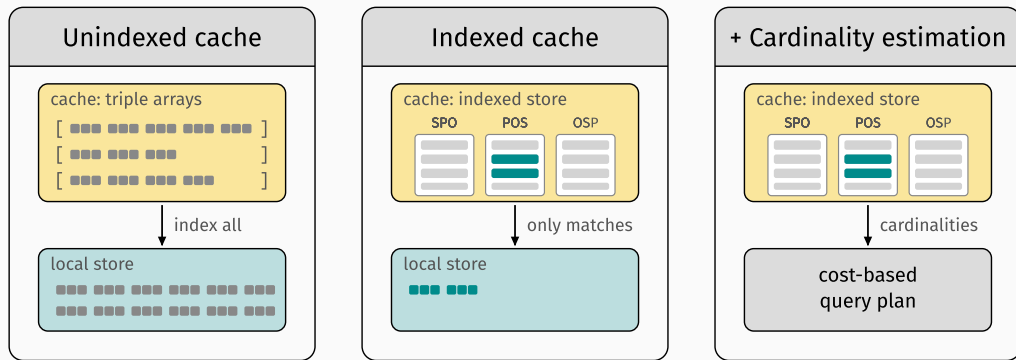
P.4 Non-selective patterns

- T.1 Predicate additions expand or contract the reachable sub-web
 - T.2 New starting points change the reachable documents
 - T.3 Changes the number of hops to relevant data (SolidBench L.1–L.6)
 - T.4 Alters usability of structural indexes, such as type indexes (L.8)
- ⇒ Custom refinement patterns can be defined in JSON

- Caching directly exploits data from past queries (Hartig 2011)
- Correlated query sequences should lead to cache hits
- Cache hit rates reveal whether our sequences behave as intended

⇒ Goal: validate the benchmark, not an exhaustive cache comparison

Caching Approaches



Implemented in Comunica (Taelman, Van Herwegen, et al. 2018), LRU eviction, three sizes: S (350k quads, ~10%), M (700k, ~20%), L (1.4M, ~40%)

Experimental Setup

Workload

- SolidBench scale 0.1: 158,233 RDF files across 1,531 vaults
- 8 generated sequences, 192 queries (discover and short templates)

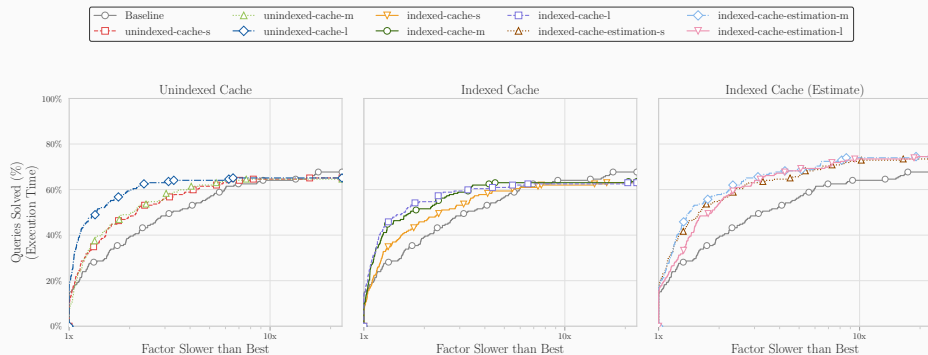
Execution

- Each sequence repeated 10 times, 70–90 ms injected network latency, 180 s timeout
- Cache scoped to a single sequence, no state shared between runs

Measured

- Cache hit rates, evictions, and speedup over no cache

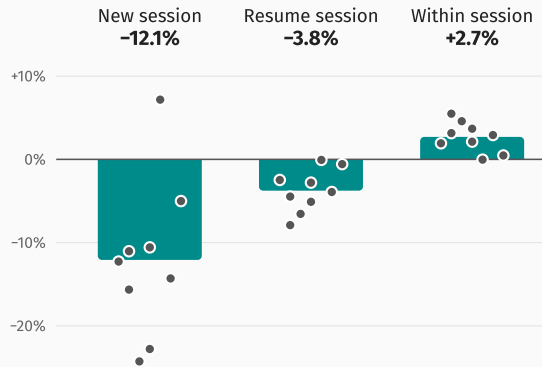
Results: All Caches Outperform No Caching



Performance profile (Dolan and Moré 2002): higher and further left is better

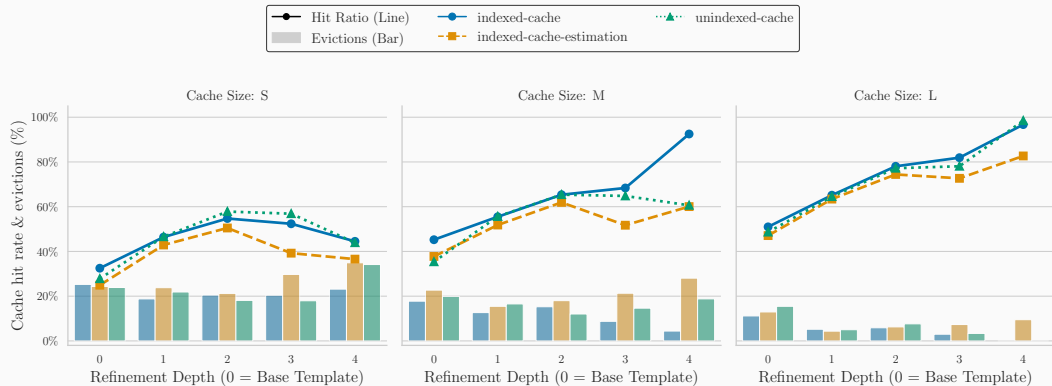
Cardinality estimation performs best and solves the most queries

Results: Session Switching Costs Cache Hits



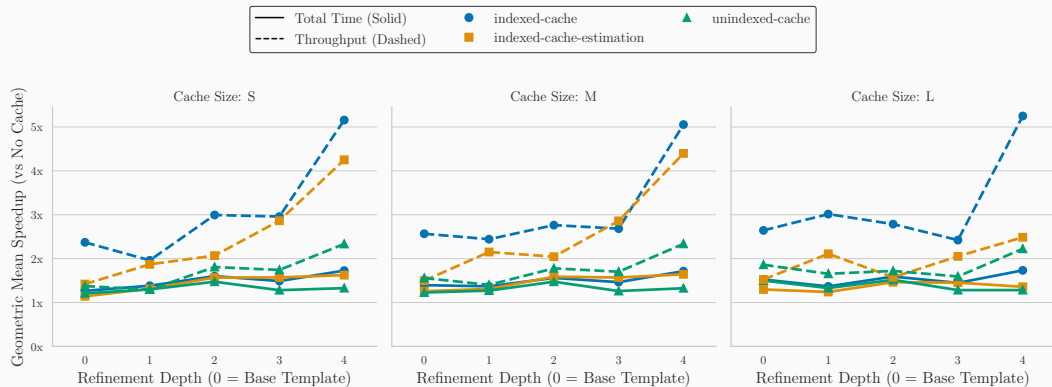
- Hit rate compared to the average of the same template
 - New sessions: -12%
 - Staying in a session: +3%
- ⇒ Cache sizing and eviction depend on session behavior

Results: Refinement Reuses Prior Data



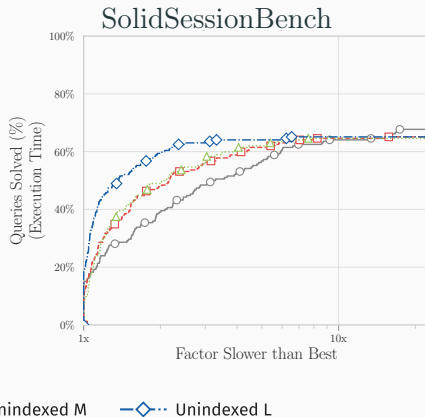
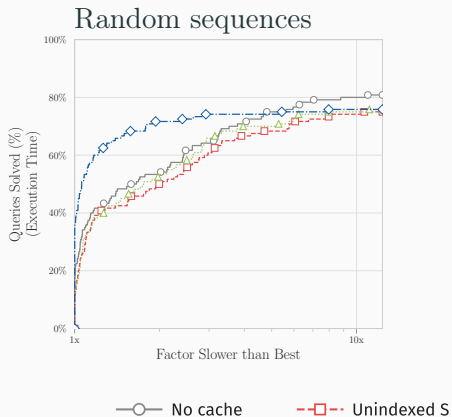
Hit rates rise with each refinement step, until small caches start thrashing

Results: Refinement Translates into Speedups



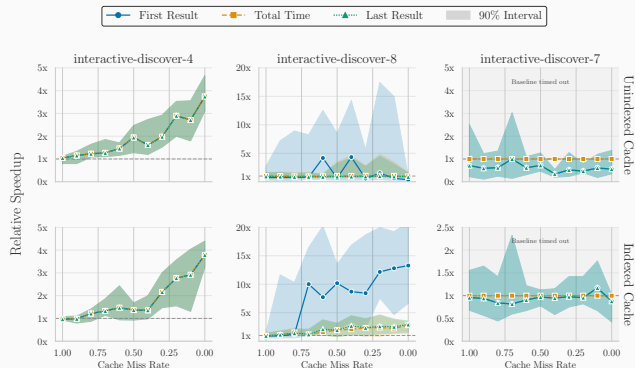
Up to $\sim 5\times$ throughput at refinement depth 4 with the indexed cache

Comparison: Random Sequences Mislead



Random: only large caches help vs. SolidSessionBench: small caches help too

Comparison: Micro Benchmarks Give Mixed Signals



- Each template run twice with simulated cache miss rates
 - 8 of 12 templates speed up with hit rate
 - Long-running queries: no speedup, or even slowdowns
- ⇒ Isolated queries do not show behavior in realistic sequences

- Realistic sequences reveal how sessions and refinement drive client-side performance
 - Random sequences and micro benchmarks miss these effects, or lead to the wrong conclusions
- ⇒ SolidSessionBench: a reproducible basis for evaluating client-side optimizations

-  Arenas, M. et al. (2025). “Exploring Exploratory Querying”. In: Proceedings of the VLDB Endowment 18.13, pp. 5731–5739.
-  Bizer, C. and A. Schultz (2011). “The berlin sparql benchmark”. In: Semantic Services, Interoperability and Web Applications: Emerging Concepts. IGI Global Scientific Publishing, pp. 81–103.
-  Bonifati, A., W. Martens, and T. Timm (2020). “An analytical study of large SPARQL query logs”. In: The VLDB Journal 29.2, pp. 655–679.
-  Dolan, E. D. and J. J. Moré (2002). “Benchmarking optimization software with performance profiles”. In: Mathematical programming 91.2, pp. 201–213.
-  Hartig, O. (2011). “How Caching Improves Efficiency and Result Completeness for Querying Linked Data”. In: LDOW.

-  Martin, M., J. Unbehauen, and S. Auer (2010). “Improving the performance of semantic web applications with SPARQL query caching”. In: Extended Semantic Web Conference. Springer, pp. 304–318.
-  Meiss, M. et al. (2009). “What’s in a session: tracking individual behavior on the web”. In: Proceedings of the 20th ACM conference on Hypertext and hypermedia. Torino Italy: ACM, pp. 173–182. DOI: [10.1145/1557914.1557946](https://doi.org/10.1145/1557914.1557946).
-  Mislove, A. et al. (2007). “Measurement and analysis of online social networks”. In: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement, pp. 29–42.
-  Mitzlaff, F. et al. (2013). “User-relatedness and community structure in social interaction networks”. In: arXiv preprint [arXiv:1309.3888](https://arxiv.org/abs/1309.3888).

-  Nicholson, H., P. Chrysogelos, and A. Ailamaki (2024). “HPCache: memory-efficient OLAP through proportional caching revisited”. In: The VLDB Journal 33.6, pp. 1775–1791.
-  Saleem, M. et al. (2015). “LSQ: the linked SPARQL queries dataset”. In: International semantic web conference. Springer, pp. 261–269.
-  Taelman, R., J. Van Herwegen, et al. (2018). “Comunica: a modular SPARQL query engine for the web”. In: International Semantic Web Conference. Springer, pp. 239–255.
-  Taelman, R. and R. Verborgh (2023). “Link traversal query processing over decentralized environments with structural assumptions”. In: International Semantic Web Conference. Springer, pp. 3–22.

Category	Parameter	Value
Log-normal (μ, σ)	Sequence length	3.0, 0.2
	Session length	1.7, 0.5
	Transition probability	-2.0, 0.25
Refinements	<code>patternProbability</code>	0.1
	<code>min/maxRefinementLength</code>	2, [3-5]
Instantiation	<code>temperature</code>	0.1
	<code>maxLogits</code>	5-10

A single seeded PRNG guarantees reproducible sequences